

Administración de memoria central y técnicas de administración de memoria

Administración de memoria central

El sistema de memoria y su administración

Por lo común el sistema de memoria de una computadora, considerando brevemente sólo a la memoria principal y a la secundaria, está organizado como un gran vector unidimensional de palabras, cada una de ellas con su propia dirección. El sistema de gestión de memoria o administrador de memoria o gestor de memoria es la porción del sistema operativo que se encarga de administrar parcialmente, no en su totalidad, la memoria, es decir, llevar el registro de cuáles partes de la memoria están siendo utilizadas, asignar memoria a los procesos cuando estos la necesiten y liberarla cuando ellos finalicen.

Sin importar cuál sea la política de administración de memoria empleada en por un sistema, se pueden destacar las siguientes características como objetivos deseables de la gestión de memoria:

- Proveerle a cada uno de los procesos un espacio lógico propio.
- Asegurar la protección entre los procesos.
- Permitir que los procesos compartan memoria.
- Dar soporte a las distintas regiones del proceso.
- Maximizar el rendimiento del sistema de memoria.
- Proporcionarle a los procesos mapas de memoria extensos.

Conviene recordar que, como en lenguaje ensamblador no hay ninguna instrucción que pueda contener direcciones en memoria secundaria, todas las instrucciones que conforman un programa en ejecución y los datos que las mismas utilizan deberán encontrarse almacenados en memoria



interna (RAM, caché, registros). En el caso de que los datos no se encuentran en esta memoria, deberán ser cargados en ella antes de que la CPU pueda disponer de ellos.

Más allá de cómo se administre la memoria, todo proceso se desplazará durante su ejecución entre la memoria secundaria, por ejemplo, en un RAID, y la memoria principal. Los procesos que están en la memoria secundaria esperando a ser cargados en memoria principal para ser ejecutados conforman la cola de entrada. Posteriormente uno de estos procesos será seleccionado por el planificador de trabajos para asignarle memoria y pasarlo a la cola de procesos listos.

Cabe destacar de entre los diferentes programas a ejecutar dos de ellos: las rutinas reutilizables y las rutinas reentrantes. *A las rutinas reutilizables se las define simplemente como rutinas que pueden ser utilizadas por dos o más tareas, mientras que las rutinas reentrantes, formadas por código reentrante, o código puro, son aquellas que podrán ser ejecutadas por varios procesos simultáneamente¹.*

El código que forma estas rutinas presenta la cualidad de ser un código que no puede modificarse a sí mismo, es decir, nunca cambia durante su ejecución. Así, dos o más procesos pueden ejecutar simultáneamente el mismo código. Como los datos locales pertenecientes a cada uno de los usuarios que están usando a la rutina, se deben almacenar en forma separada, cada proceso tendrá su propia copia de los registros y del almacenamiento de datos que serán usados durante la ejecución del proceso. Obviamente, dos procesos distintos dispondrán de datos diferentes. Otra característica que destacar es que una rutina de esta clase puede ser interrumpida e invocada por el proceso que generó la interrupción para luego ser ejecutada correctamente cuando se complete la interrupción y se vuelva a la rutina.

Todo programa de código puro tendrá una parte inmodificable formada por las instrucciones que lo componen y una parte transitoria formada por un puntero de regreso al proceso que realizó la invocación más un espacio para las todas las variables locales usadas por el proceso. En toda activación o ejecución de la rutina se ejecutará el código en la sección inmodificable junto con su copia exclusiva de los parámetros y las variables locales. La parte transitoria relacionada con una activación específica se llama registro de activación que se guarda en la pila del procesador cuando la rutina es invocada. Como ejemplo de estas rutinas se puede mencionar a las rutinas que forman el núcleo del sistema operativo.

¹ Mataix Lorda, M. y Mataix Hidalgo, M. (1999). *Diccionario de electrónica, informática y energía nuclear*. Madrid: Ediciones Díaz de Santos S.A.

Asociación de direcciones

Desde que se implementó la arquitectura de von Neumann la asociación entre las instrucciones y los datos con las direcciones de memoria se puede realizar en cualquiera de las siguientes etapas:

Tiempo de compilación

Si en este momento se sabe dónde residirá el proceso en memoria, entonces se puede generar el código absoluto. Es decir que, si se sabe que el proceso estará en memoria a partir de la dirección M1, entonces el código absoluto generado por el compilador comenzará en esa dirección de memoria y se extenderá hasta su dirección final M2. Más adelante si, por el motivo que sea, esta posición de inicio cambia, entonces será necesario recompilar el código.

Tiempo de carga

Si no se sabe dónde residirá en memoria el proceso en el momento de su compilación, entonces el compilador puede generar un código reubicable. Por lo tanto, el enlace entre las instrucciones, los datos y las direcciones de memoria donde residirán se demora hasta el tiempo de carga. Ahora, si la posición de comienzo cambia, solo se necesita cargar nuevamente el código para incorporar el nuevo valor.

Tiempo de ejecución

En el caso de que el proceso se pueda desplazar, o que sea removido, durante su ejecución desde un lugar de memoria a otro, entonces será necesario demorar el enlace hasta el momento de su ejecución. Como condición para que este esquema se pueda implementar, será necesario que el hardware tenga una unidad de administración de memoria (MMU) que pueda llevar a cabo esta tarea. Como todos los procesadores actuales disponen de ella, la mayoría de los sistemas operativos de propósito general utilizan este método.

Espacio de direcciones lógicas y físicas

Tal como se indica en todo curso de arquitectura de computadores, toda dirección de memoria generada por el procesador se llama generalmente dirección lógica, mientras

que una dirección de memoria guardada en el MAR (*Memory Address Register*) y utilizada por la memoria interna se llama usualmente dirección física.

Tanto el enlace de direcciones en tiempo de compilación como el enlace en tiempo de carga generan direcciones tanto lógicas como físicas, es decir, son iguales. Pero, el enlace de direcciones en tiempo de ejecución provoca que las direcciones lógica y física sean divergentes, por lo que, habitualmente se habla de una dirección virtual en lugar de una dirección lógica.

Todo programa genera un conjunto de direcciones lógicas las cuales forman un espacio de direcciones lógicas. Pero, además, este espacio de direcciones lógicas tiene asociado un conjunto formado por todas las direcciones físicas que le corresponden y recibe el nombre de espacio de direcciones físicas. Luego, en el enlace de direcciones en tiempo de ejecución resulta que ambos espacios divergen y la tarea de establecer la correspondencia entre esas dos clases de direcciones, virtuales y físicas, recaerá sobre un componente del núcleo del procesador llamado unidad de administración de memoria (*Memory Management Unit*) que contiene un registro de reubicación cuyo contenido es sumado a la dirección lógica para así generar la dirección física.

Carga dinámica

Si se desea implementar la idea original de una computadora de programa almacenado según la planteó von Neumann, la totalidad del programa junto con la totalidad de los datos de un proceso deben estar cargados en memoria principal para que ese proceso pueda ejecutarse, por lo que, el tamaño de un proceso no podrá exceder el tamaño de la memoria principal. Sin embargo, si se quiere aprovechar al máximo el espacio en una memoria principal de capacidad limitada, se puede implementar la carga dinámica. Con ella, un programa se carga en memoria sin las rutinas, las cuales son mantenidas en disco en un formato de carga reubicable. Durante el proceso, cuando una rutina es necesaria, se verifica si ya está cargada, sino es así se invoca al cargador de montaje reubicable para que la traiga a memoria y para que actualice las tablas de direcciones del proceso para registrar el cambio. Posteriormente se le transfiere el control a la rutina recién cargada.

El beneficio de esta política de trabajo radica en que toda rutina que no es invocada jamás será cargada en memoria, siendo esto muy conveniente cuando se utilizan programas muy grandes para tratar eventos que ocurren en forma esporádica. En esta situación, por

más grande que sea el programa, la parte que se usa en un determinado instante, es mucho menor.

Otro beneficio que presenta la carga dinámica es que no necesita apoyo especial del sistema operativo, sino que queda a cargo del programador diseñar sus programas para usar este mecanismo, aun cuando los sistemas operativos suelen ayudar al programador suministrándole rutinas de biblioteca que permiten implementar la carga dinámica.

Enlace dinámico y estático de librerías

Conviene definir primeramente el concepto de librería. Una librería es un conjunto de objetos relacionados, o no, entre sí, pertenecientes al sistema operativo, o creados por el usuario, que les brindan diferentes servicios a los programas.

Una forma empleada para la creación de un ejecutable comienza con la compilación de los módulos fuente de la aplicación, para que luego el cargador integre los módulos objeto obtenidos con los extraídos de las librerías que correspondan dentro de la imagen binaria del programa. Así, el ejecutable incluye todo el código que necesita el programa para poder ser ejecutado. Esta forma de trabajo se denomina enlace o montaje estático de librerías. De esta manera todos los programas necesitan tener una copia de su librería de lenguaje incluida en su programa ejecutable. Esta exigencia genera un pésimo aprovechamiento del espacio tanto en el disco como en la memoria principal.

Para superar esta situación se utiliza una técnica denominada enlace o montaje dinámico de las librerías que consiste en postergar el enlace con los módulos externos hasta el momento en que se crea el módulo de carga. Luego este módulo de carga contendrá referencias a rutinas de librería que no están resueltas y que se podrán solucionar ya sea en el tiempo de carga o de ejecución. Esto da lugar a dos formas del montaje dinámico: la implícita y la explícita. Cuando se especifica en tiempo de montaje cuáles son las rutinas de librerías que se usarán y se pospone la carga y el montaje correspondiente hasta el tiempo de ejecución el montaje recibirá el nombre de montaje dinámico implícito. Pero, cuando se desconoce en tiempo de montaje qué librerías se necesitarán y por lo tanto se requiere poder establecer en tiempo de ejecución que librería dinámica se necesita para pedir explícitamente su montaje y carga se está ante lo que se llama un montaje dinámico explícito.

En cualquiera de estos casos se incluirá dentro de la imagen binaria del programa un pequeño código que dirá cómo ubicar a la rutina de librería residente en memoria o cómo

cargar dicha librería si esta no está todavía presente en la memoria. Cuando este código se ejecuta comenzará por comprobar si la rutina correspondiente ya está cargada en memoria. Si no lo está, entonces este código hará que el cargador ubique a la rutina y la cargue en la memoria. Una vez que la rutina está efectivamente en la memoria, el código se reemplazará así mismo por la dirección de la rutina y ejecutará a la rutina. De esta forma, la próxima vez que se ejecute ese fragmento del código del programa, se ejecutará directamente la rutina de librería, sin tener que realizar otra vez el montaje dinámico. Con esta técnica, todos los procesos que utilicen una librería de lenguaje determinada sólo necesitarán ejecutar una copia del código de la librería.

Cabe acotar que el montaje dinámico suele necesitar ayuda del sistema operativo porque, dada la protección en memoria que éste le brinda a los procesos, entonces el sistema operativo será el único que pueda comprobar si la rutina buscada se encuentra dentro del espacio de memoria de otro proceso y, en tal caso, será el único que permita a varios procesos acceder a esa sección de la memoria.

Swapping (intercambio)

El swapping es una técnica que permite intercambiar temporalmente un proceso cargado en memoria transfiriéndolo a un almacenamiento secundario, generalmente un disco (*swap out*), para posteriormente traerlos otra vez a memoria principal para continuar su ejecución (*swap in*).

El lugar donde un proceso es puesto nuevamente al hacer *swap in*, dependerá si la asociación de direcciones fue hecha en tiempo de compilación, de carga, o en tiempo de ejecución. Si ocurrió en tiempo de compilación o carga entonces deberá ser cargado en el mismo espacio de memoria de donde fue quitado, pero si fue en tiempo de ejecución no importará donde es cargado nuevamente.

Otra cuestión importante que destacar es la siguiente: si el intercambio se realiza entre memoria principal y disco, ¿en qué momento se reserva el espacio en el disco para hacer el intercambio? Para este interrogante existen dos respuestas posibles:

1. El espacio de swap suficiente para albergar al proceso es reservado en el momento en que el proceso es creado. Esta modalidad recibe el nombre de preasignación.
2. La segunda posibilidad consiste en reservar el espacio de *swap* suficiente para albergar al proceso sólo cuando se desactiva al proceso. A esta modalidad se la llama sin preasignación.

Aunque muchas versiones de Unix siguen utilizando una versión optimizada de esta técnica, en la actualidad se la utiliza muy poco porque el tiempo de intercambio es bastante grande, además de que sea un proceso cuyo el tiempo de ejecución sea muy superior en relación al tiempo de intercambio para lograr un buen uso del procesador o que el proceso a intercambiar esté completamente inactivo.

Overlays (superposiciones)

Esta técnica, usada cuando el espacio en memoria era extremadamente limitado, consistía en dividir un programa en secciones que se ejecutaban en forma sucesiva manteniendo en memoria principal solo la sección del programa y los datos que se necesitaban en un momento determinado.

Cada sección era programada de manera que después de realizar su labor cargaba en memoria, en el mismo lugar donde residían otras instrucciones que ya no se necesitaban, la siguiente sección y le cedía el control.

A esta forma de gestionar la memoria, el sistema operativo no le brindaba soporte alguno, por lo que era el programador quien debía diseñar y programar las superposiciones adecuadamente. En la actualidad, dadas estas limitaciones, más el desarrollo de las técnicas de memoria virtual, la implementación de *overlays* cayó en desuso, limitándose sólo al campo de los microcomputadores.

Fragmentación de la memoria

La fragmentación de la memoria es un problema que se presenta en algunas de las políticas de administración de la memoria. Este problema se presenta bajo dos formas diferentes: fragmentación interna y fragmentación externa:

- La fragmentación interna se presenta cuando una porción del espacio de memoria que es asignado a un proceso no es utilizada por el mismo y se desperdicia.
- En el caso de la fragmentación externa existe un espacio total de memoria libre suficiente para satisfacer el requerimiento de un proceso. Pero, como dicho espacio no es contiguo, sino que está dividido en fragmentos de tamaño reducido, resulta imposible asignarlos a un proceso.

Una posible solución a esta forma de fragmentación consiste en compactar la memoria, es decir, mover todo el contenido de la memoria asignada a los procesos para que queden

contiguos y así también lograr que todos los espacios libres se unifiquen en un único gran bloque. Su aplicación en general se hace a través de *roll out* y *roll in* y sólo si la relocalización es dinámica y hecha en tiempo de ejecución.

Técnicas de administración de memoria

Esquemas de memoria basados en asignación contigua

Se pueden destacar los siguientes esquemas de gestión de memoria consisten siempre en asignar a cada proceso una zona contigua de memoria. De esta forma se logra el objetivo de que en ella esté su mapa de memoria:

Esquema de asignación continúa

Este esquema de gestión de la memoria presenta las características siguientes:

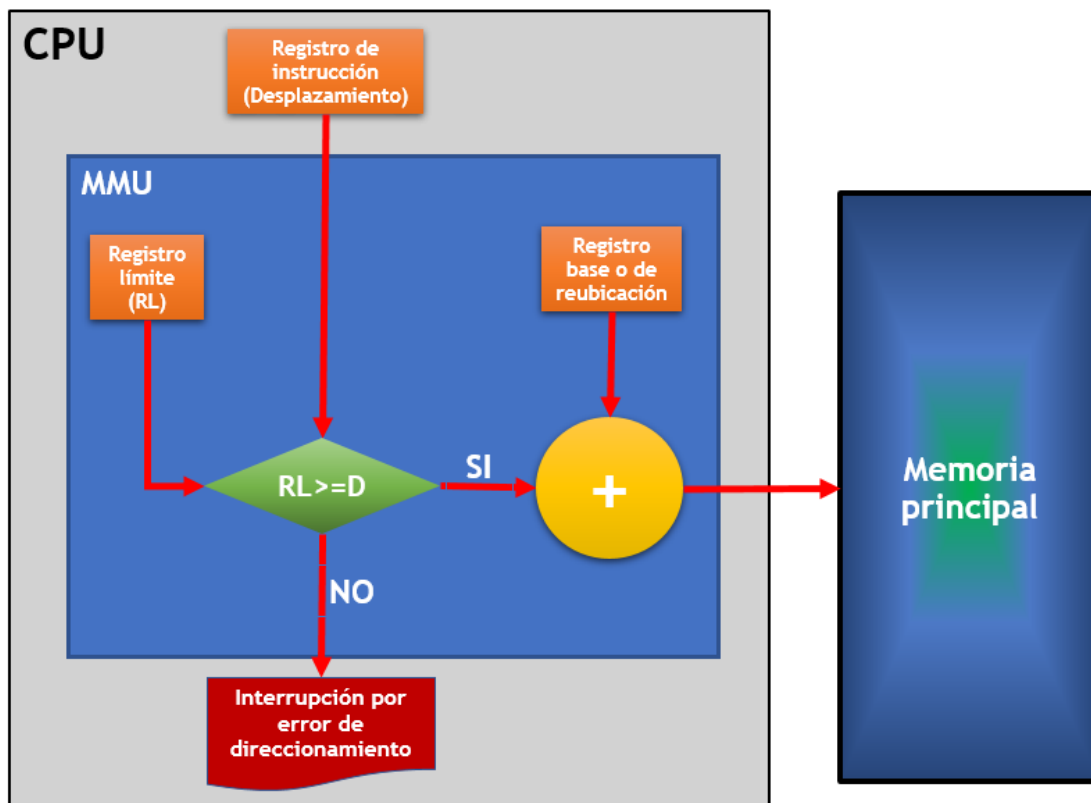
- Al usuario se le proporciona la máquina básica y tiene el control absoluto sobre todo el espacio de memoria.
- Ofrece al usuario máxima flexibilidad, es sencillo y su costo es mínimo puesto que no requiere de un hardware especial, pero no ofrece servicios ya que no tiene sistema operativo.
- Se utiliza en sistemas dedicados donde los usuarios requieren flexibilidad y sencillez, y están dispuestos a programar sus propias rutinas de apoyo.

Esquema de asignación mediante partición simple

En este diseño la memoria principal almacenará tanto al sistema operativo como al proceso del usuario. El sistema operativo podrá estar bien en la zona baja o en la zona alta de la memoria dependiendo esto de la ubicación del vector de interrupciones ya que, por lo general, este es ubicado en la parte baja de la memoria porque los programadores suelen situar al sistema operativo en dicha zona. Cabe acotar que la mayoría de los estudios que se realizan sobre el tema suponen que el sistema operativo reside en la parte baja de la memoria, aunque los análisis hechos por esos estudios son totalmente aplicables para al otro caso.

Teniendo esto presente, los esquemas de asignación mediante partición simple suelen tener las siguientes características:

- Dividen la memoria en dos partes: una para el sistema operativo y otra para el usuario.
- Son usados principalmente en sistemas de monoprogamación.
- El diseño necesita de un hardware de manejo de memoria relativamente simple, usualmente formado por dos registros valla, el registro base y el registro límite, únicamente accesibles en modo privilegiado, que se usarán para tratar cada una de las direcciones que genera un programa con el fin de para impedir accesos ilegales a la memoria.
- En el siguiente gráfico el registro base actúa como un registro de reubicación mientras que la instrucción contiene el desplazamiento (D) y el registro límite (RL) contiene el mayor desplazamiento permitido. Luego, el conjunto trabaja de la siguiente forma con el objetivo de impedir accesos ilegales a la memoria principal por la generación de direcciones que no pertenecen al espacio de direcciones del proceso. Como se puede apreciar en el gráfico, si el valor límite no supera o iguala al valor del desplazamiento (D) que está en el registro de instrucción, se generará una interrupción por error de direccionamiento. En cambio, cuando si lo hace, se sumará ese desplazamiento al registro base para lograr la dirección de memoria física deseada.



Esquema de asignación de particiones múltiples de tamaño fijo

Si en la memoria principal se deberán ubicar el sistema operativo residente junto con los diferentes procesos de usuario, entonces, será imprescindible asignar y gestionar la memoria principal de la mejor forma posible. Resultará necesario tener presente que esta memoria se dividirá en dos partes fundamentales, una parte para el sistema operativo residente y la otra para los procesos de usuario.

El esquema de asignación de particiones múltiples de tamaño fijo es la forma más sencilla de administrar la zona de la memoria principal reservada para los usuarios. Consiste en dividirla esta zona en particiones, ya sean todas del mismo tamaño o de diferente tamaño, pero siempre de un tamaño permanente. El número de estas particiones establecerá el mayor grado de multiprogramación posible puesto que, en principio, en cada partición se ubica un único proceso.

En cuanto a la pregunta de cómo se asignaban las particiones, son posibles dos respuestas:

- Si todas las particiones tenían el mismo tamaño entonces, no bien había una partición disponible, se cargaba un proceso en ella sin importar cuál partición era. En este diseño también podía ocurrir que, de ser necesario, cualquiera de los procesos que ocupaban una partición y no estaban en estado listo podía ser seleccionado para realizar un intercambio a disco y así disponer de su partición para cargar en ella otro proceso de mayor prioridad.
- En cambio, si las particiones eran de distinto tamaño, las mismas se podían asignar de dos formas distintas. Una consistía en asignar cada proceso a la partición más pequeña que lo pudiese alojar. Luego era necesario tener una cola de planificación para cada partición con todos los procesos que se le podían asignar a la partición. La otra forma consistía en usar una única cola para todos los procesos de manera que, cuando era el tiempo de cargar un proceso en la memoria principal, se elegía la partición más pequeña disponible en la cual el proceso se pudiese alojar. Al igual que en el caso de las particiones de igual tamaño, de ser necesario se elegía un proceso para realizar un intercambio a disco y así disponer de su partición.

Este esquema que usó en sistemas de IBM bajo OS/360 llamado MFT, y que en la actualidad ya no es usado, tenía dos grandes desventajas:

- Un proceso podía ser demasiado grande para caber en una partición por lo que el programador estaba obligado a usar *overlays* en el diseño del programa para que fuese posible su ejecución.
- Como siempre cada proceso debía ocupar una partición entera se tenía un uso de la memoria principal es muy ineficiente generando mucha fragmentación interna.

Esquemas de asignación de particiones múltiples de tamaño variable

Con el fin de superar varias de las dificultades del esquema de particiones múltiples de tamaño fijo se desarrolló una técnica conocida como asignación de particiones múltiples de tamaño variable o dinámico que presentaba las siguientes características:

- El sistema operativo mantenía una tabla que indicaba las particiones que estaban libres y las que estaban ocupadas.
- Inicialmente consideraba a toda la memoria disponible para los procesos de usuario y formaba un solo bloque o hueco.
- Se usó para procesos *batch*.
- Asignaba lugar a los procesos en función de los huecos libres existentes.

Su forma de trabajo consistía en asignarle a todo proceso que se quería cargar en memoria principal un hueco (espacio libre en memoria) lo suficientemente grande como para poder ubicar allí al proceso. Si podía encontrarlo sólo le asignaba la cantidad exacta de memoria contigua pedida, reservando el resto de la memoria disponible para poder cumplir con futuras demandas. Por el contrario, si no había un hueco disponible que fuese lo suficientemente grande como para albergar al proceso, entonces el sistema operativo esperaba hasta que hubiese una partición libre lo suficientemente grande para alojar al proceso o revisaba el resto de la cola de entrada para ver si se podía cumplir con los requerimientos de memoria de algún otro proceso.

Por supuesto que, en todo momento, el sistema operativo tenía presente las necesidades de memoria de cada uno de los procesos y cuánta memoria libre existe cuando debía elegir el o los procesos a los que le asignaba esa memoria disponible. Para simplificar esta elección, el sistema operativo podía ordenar la cola de procesos de entrada según alguna forma de planificación. Luego asignaba la memoria siguiendo el orden establecido hasta que no podía cumplir con las necesidades de un proceso de la cola porque no existía un hueco con el tamaño necesario como para cargar allí a ese proceso. En ese instante, el sistema operativo tenía dos opciones: esperar hasta que se libere un espacio de memoria lo bastante extenso como para ubicar allí al proceso en cuestión, o examinar el resto de la

cola de entrada para ver si podía encontrar un proceso que necesitase un hueco de menor tamaño.

Por lo general, los espacios de memoria disponible, es decir los huecos, se encontraban desperdigados a todo lo largo de la memoria, por lo que, cuando un proceso nuevo necesitaba memoria, el sistema operativo busca un hueco que sea lo suficientemente grande como para colocar allí al proceso. Si el hueco seleccionado era muy grande, el sistema operativo lo dividía en dos particiones, asignándole una de las particiones al proceso que cargaría en la memoria y juntando la otra con el resto de los huecos. Como es de esperar, al finalizar el proceso liberaba la partición que le había sido asignada y la misma se sumaba al resto de los huecos. Luego si la partición recién liberada era contigua a otros huecos todos ellos se unían para generar uno más grande. Y la historia vuelve a empezar, es decir, el sistema operativo nuevamente busca en la cola de procesos de entrada si existe algún proceso al que le podía asignar este nuevo espacio de memoria disponible.

Ahora bien, aunque la técnica empieza sin problemas, en algún momento termina con que el espacio libre de la memoria está repartido en una infinidad de pequeños huecos y cuanto más pase el tiempo más se fragmenta la memoria y su funcionamiento se deteriora. Esta situación recibe el nombre de fragmentación externa en contraposición a la fragmentación interna, ambas ya descritas en el módulo anterior.

Ante esta situación, al diseñador del sistema operativo le quedan dos opciones: programar al sistema para que ante un nivel de fragmentación muy elevado se realice automáticamente una compactación o aplicar alguna de las políticas de asignación del espacio libre en la memoria principal. La primera de estas opciones tiene varias desventajas: consume mucho tiempo del uso de la memoria, malgasta el tiempo de uso del procesador y requiere indefectiblemente que los procesos sean dinámicamente reubicables caso contrario no se puede aplicar. Dada esta situación la segunda de las opciones resulta más interesante ya que el diseñador sólo debe elegir una política de asignación. Tres son las políticas usualmente aplicadas: primer ajuste, mejor ajuste y peor ajuste.

Primer ajuste

Se asigna el proceso al primer hueco en el cual quepa buscando desde el comienzo de la lista de huecos, logrando que este algoritmo sea rápido porque busca lo menos posible. Una variante de esta política es la política del **siguiente ajuste** que funciona de la misma

manera que el primer ajuste, pero que memoriza dónde se encuentra cada vez que encuentra el hueco adecuado. De esta forma la próxima vez que se debe buscar un hueco, la misma comienza a partir del lugar en el que se asignó el último hueco, en lugar de comenzar siempre desde el principio de la lista de huecos.

Mejor ajuste

El proceso se ubica en el hueco en el que mejor se ajusta. Aunque en un primer análisis ésta puede parecer la mejor política, en realidad no lo es. Primero porque tiende a generar nuevos huecos muy pequeños y segundo porque la búsqueda del mejor hueco exige comparar con cada uno de sus tamaños o mantener la lista de huecos ordenada por los tamaños. Y esto torna ineficiente a esta política.

Peor ajuste

El proceso se asigna al hueco más grande disponible donde quepa. Su principal objetivo es el de impedir la aparición de nuevos huecos pequeños, aunque tenga necesariamente que recorrer toda la lista de huecos u ordenarla en forma creciente o decreciente por tamaño lo cual hace que esta política sea tan ineficiente como la del mejor ajuste.

Aunque el determinar cuál de las técnicas es la mejor depende de una secuencia precisa de intercambio de los procesos y del tamaño de estos, se puede decir que la estrategia del primer ajuste es la más sencilla y rápida de implementar como parte del administrador de memoria.

Ejemplo 1

Considerar un sistema que trabaja con un esquema de administración de memoria con particiones múltiples y tiene disponibles las particiones que se detallan más abajo. Al mismo llegan en orden los procesos A, B, C y D que requieren respectivamente: 160KB, 420KB, 280KB y 350KB. Se verá, para cada uno de los tres esquemas de asignación planteados, cómo se asignarían las particiones, si alguno deja suspendido uno o varios procesos y cuál es la fragmentación externa de cada caso:

N.º de partición	Tamaño en KB	Mejor ajuste		Primer ajuste		Peor ajuste	
		Proceso asignado	Fragmentación interna	Proceso asignado	Fragmentación interna	Proceso asignado	Fragmentación interna
1	360	D	10 KB	A	200 KB	C	80 KB
2	180	A	20 KB	---		---	
3	300	C	20 KB	C	20 KB	---	

4	520	B	100 KB	B	100 KB	B	100 KB
5	620	--		D	270 KB	A	460 KB
Proceso/s suspendido/s por falta de espacio en memoria		---		---		D	
Fragmentación externa en KB		620 KB		180 KB		480 KB	

Ejemplo 2

Considerar un sistema que trabaja con un esquema de administración de memoria con particiones múltiples variables que tiene disponibles las particiones que se detallan en la siguiente figura. Al mismo llegan en orden alfabético los procesos que se indican a continuación los cuales requieren los siguientes espacios en memoria: Proceso A de 210 KB, Proceso B de 550 KB, Proceso C de 320 KB y Proceso D: 190 KB. Se verá, para cada uno de los tres esquemas de asignación planteados, cómo se asignarían las particiones, si alguno deja suspendido uno o varios procesos y cuál es la fragmentación externa de cada caso:

N.º de partición	Tamaño en KB	Mejor ajuste		Primer ajuste		Peor ajuste	
		Proceso asignado	Fragmentación interna	Proceso asignado	Fragmentación interna	Proceso asignado	Fragmentación interna
1	290	--		A	80 KB	D	100 KB
2	370	C	50 KB	C	50 KB	C	50 KB
3	250	A	40 KB	D	60 KB	---	
4	200	D	10 KB	--		---	
5	580	B	30 KB	B	30 KB	A	370 KB
Proceso/s suspendido/s por falta de espacio en memoria		----		---		B	
Fragmentación externa en KB		290 KB		200 KB		450 KB	

Paginación

Todos los métodos de administración de memoria vistos hasta ahora presentan cierto grado de ineficiencia administrando la memoria ya sea porque los primeros generan fragmentación interna mientras que los últimos generan fragmentación externa. Con el fin de superar esta situación la paginación plantea la siguiente idea: abandonar el requerimiento de asignación de memoria contigua y dividir tanto la memoria principal como los procesos en secciones de tamaño fijo relativamente pequeñas. A las secciones del proceso, llamadas páginas, o páginas lógicas, se les asigna las secciones libres de la memoria, llamadas marcos, o marcos de páginas o páginas físicas o *frames* físicos.

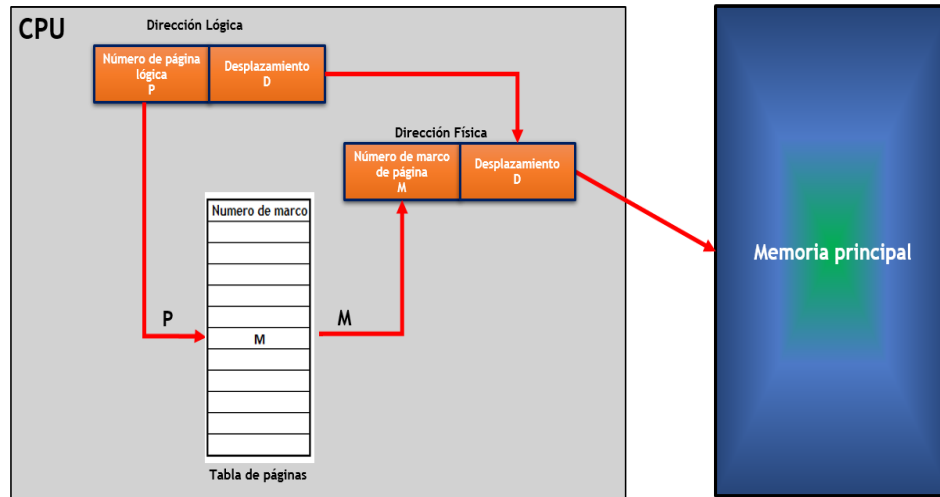
Luego, en un sistema de memoria paginado, antes de que un proceso sea pasado al estado listo debe indicar la cantidad de páginas lógicas que tiene, si esa cantidad de *frames* libres está disponible, entonces el proceso es cargado a memoria y es puesto en la cola de procesos listos. Como resultado de la implementación de esta técnica, el espacio de memoria perdido por cada proceso a causa de la fragmentación interna es sólo una pequeña parte de la última página física asignada a un proceso y en promedio es de media página. Además, su otro beneficio es que casi no genera fragmentación externa.

Como se sabe, es el sistema operativo el que está administrando la memoria física y por lo tanto tiene que conocer todos los detalles vinculados con la asignación de una memoria física que está paginada, es decir, debe saber en todo momento cuáles marcos están asignados, cuáles están disponibles, la cantidad total de marcos, además de un sinnúmero de cuestiones relacionadas. Toda esta información, por lo general, se guarda dentro de las siguientes estructuras de datos: la tabla de páginas y la tabla de *frames* o marcos. ¿Qué características destacadas se pueden mencionar de cada una de ellas?

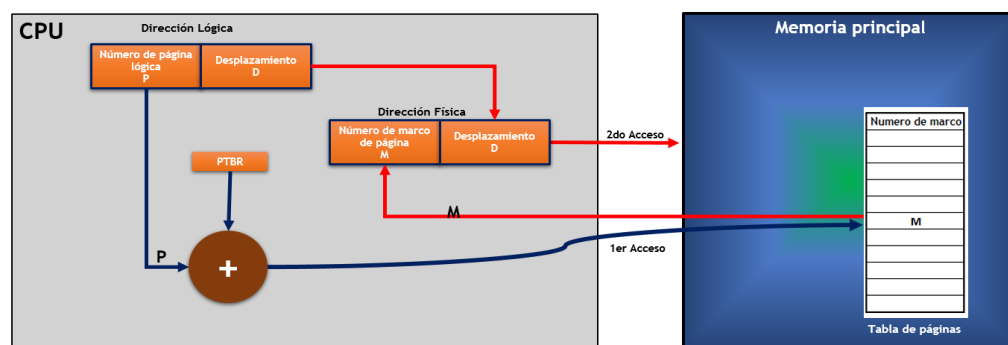
En el caso de la tabla de páginas hay que destacar lo siguiente:

- Aunque cada sistema operativo define su forma de administrar la tabla de página, en la mayoría de ellos existe una tabla por cada proceso cargado en memoria. Cada vez que un proceso entrega una dirección lógica, dicha dirección deberá ser convertida para obtener la dirección física correcta. Para lograr este objetivo el sistema operativo utiliza la tabla de páginas del proceso para traducir las direcciones lógicas a direcciones físicas. Por lo cual, cada vez que a un proceso se le entregue el procesador, el despachador debe cargar, total o parcialmente, una copia de esta tabla en los registros o en la memoria interna del procesador.

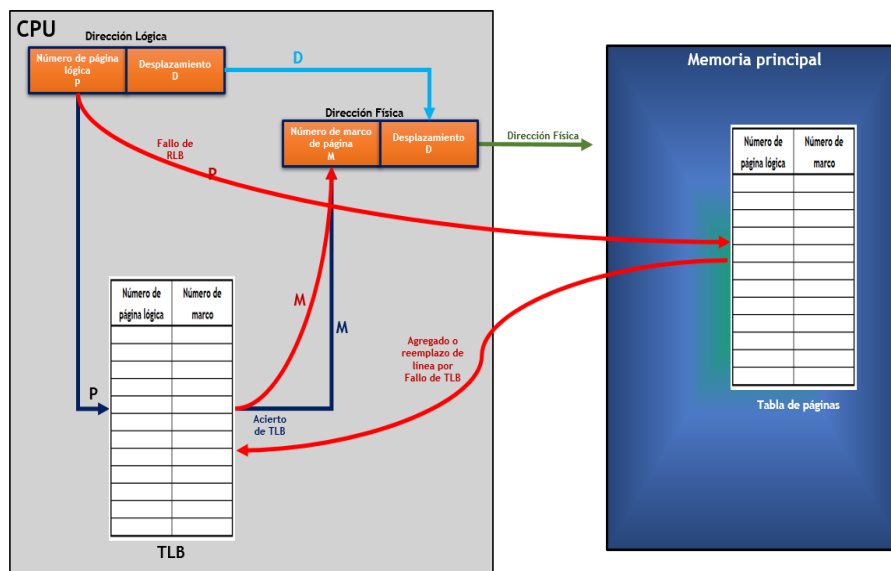
- La existencia de esta tabla para cada uno de los procesos asegura que ningún proceso acceda al espacio de otro.
- Su implementación puede realizarse de distintas maneras:
 - La forma más sencilla consiste en usar un conjunto de registros de alta velocidad especializados donde colocar la tabla para que la traducción de direcciones sea por demás eficiente. Este modelo funciona sólo si la tabla de páginas es pequeña, por ejemplo, de 512 páginas.



- Otra forma de implementación plantea mantener la tabla de páginas en memoria principal, utilizándose un registro base de la tabla de páginas (PTBR: *Page-Table Base Register*) para apuntar a la tabla de páginas. Luego cuando se necesita cambiar las tablas de páginas durante el cambio de contexto, sólo basta con cambiar el contenido de este único registro. Aunque funciona con tablas muy extensas de más de un millón de páginas, su defecto es que requiere de dos accesos a memoria, uno para acceder a la tabla, obtener el número del marco de página y generar la dirección física y el otro para acceder a la dirección física deseada.



- La siguiente implementación plantea el uso de una memoria caché asociativa con capacidad de acceso por contenido, denominada búfer de traducción anticipada de direcciones (TLB: *Translation Lookaside Buffer*). Esta memoria contendrá sólo una pequeña cantidad del total de líneas que tiene la tabla de páginas que, por lo general corresponden a las últimas páginas lógicas referenciadas. Cada una de las líneas del TLB estará conformada por dos partes: una clave (número de página lógica) y un valor (número del marco de página).



Cuando la CPU genera una dirección lógica, el número de página que la compone se envía al TLB. Si se encuentra allí ese número de página, el número de marco de página correspondiente estará inmediatamente disponible y se utilizará para conformar, junto con el desplazamiento, la dirección física. Pero si el número de página lógica no se encuentra en el TLB, es decir un fallo de TLB, será imprescindible acceder a la memoria principal para consultar la tabla de páginas completa y así obtener el número del marco de página deseado para poder armar la dirección física correspondiente. En este caso, si hay espacio en el TLB, se podrán agregar el número de página lógica y el número de marco de página. En cambio, si el TLB está lleno, será necesario realizar el reemplazo de alguna de sus líneas aplicándose entonces las mismas políticas de reemplazo que se usan en los reemplazos del sistema de cache. Cabe acotar que, algunos diseños de TLB's excluyen de la política de reemplazo a todas las líneas que corresponden al *kernel* del sistema operativo.

Establecida esta estructura se plantea lo siguiente: ¿qué pasa cuando ocurre un cambio de contexto? Son dos las posibilidades. Una de ellas consiste en que el TLB pueda guardar en cada una de sus líneas un identificador del espacio de direcciones del proceso al cual corresponde la línea. De esta forma se estaría protegiendo el espacio de direcciones de ese proceso porque la lógica del TLB chequearía que el identificador de la línea a la que se quiere acceder coincide con el del proceso que se está ejecutando en ese momento. Si no coinciden se considera que ocurrió un fallo de TLB.

La otra posibilidad es que el TLB no pueda usar esos identificadores. Luego será obligatorio en cada cambio de contexto eliminar completamente el contenido del TLB para poder garantizar que el proceso a ejecutar no esté usando una información errónea ante cada traducción de direcciones. Si no fuese obligatorio se correría el riesgo de usar información de una tabla de páginas que no pertenece al proceso en ejecución.

- Respecto de cómo está estructurada, las opciones son las siguientes:
 - Tabla de páginas jerárquica o tabla de páginas multinivel: esta opción se base en dividir la tabla de páginas en otras dos tablas, paginando la tabla de páginas, creando una tabla de páginas externa o de nivel 1 y otra interna o de nivel 2. En este esquema la traducción de las direcciones se realiza comenzando por la tabla de páginas de nivel 1 para luego continuar con la de nivel 2. En función de esta idea la dirección lógica tendrá su número de página dividido en dos partes, con lo que se tendría una terna (A, Ω, Δ) , donde A es el índice de la tabla de páginas nivel 1, Ω es el índice dentro de la página de la tabla de páginas nivel 2 y Δ es el desplazamiento dentro de la página lógica del proceso. En esta forma de estructurar la tabla de páginas para un espacio lógico con direcciones de 32 bits, también existe la posibilidad de crear una jerarquía de 3 ó 4 niveles.
 - Tabla *hash* de páginas: utilizada para administrar espacios lógicos con direcciones lógicas de memoria mayores a 32 bits esta opción se basa en el uso de diferentes técnicas hash para la implementación de una tabla hash de páginas. Cada línea de la tabla hash es una lista enlazada de elementos cuyo valor hash es el mismo. Los elementos están formados por el número de página lógica, el número del marco de página y el puntero al siguiente elemento de la lista. La idea es la siguiente. Cuando un proceso genera una dirección lógica, a su número de página lógica se le aplica una función hash para obtener un valor que se usa como índice de la tabla. Luego se compara al número de página

lógica de la dirección lógica con el número de página lógica del primer elemento de la lista enlazada. Si coincide, se obtiene el correspondiente número de marco para formar luego la dirección física. Si no coincide se pasa al siguiente valor de la lista y se repite el proceso hasta lograr la coincidencia.

- Tabla de páginas invertidas: bajo esta estrategia se diseña una única tabla de páginas con tantas líneas como páginas físicas tenga la memoria de forma que la línea número A corresponde al marco número A. En una de sus tantas implementaciones, cada una de sus líneas es formada por un código identificador del proceso y un número de página lógica. Por su parte las direcciones lógicas de memoria que se generan en el procesador constan de tres elementos: el código identificador del proceso, el número de página lógica y el desplazamiento.

Entonces, cuando un proceso genera una dirección lógica de memoria, se busca el par (código identificador del proceso, número de página lógica) en la tabla de páginas invertida. El número de línea de la tabla donde se produce la coincidencia (que será única) es el número del marco de página donde está la página lógica buscada. El valor encontrado, junto con el desplazamiento, formarán la dirección física. Como esta búsqueda puede demandar demasiado tiempo suele utilizarse una tabla hash para restringir la búsqueda a una línea sola o a unas pocas de ellas.

Mientras que en el caso de la tabla de marcos se puede mencionar lo siguiente:

- Existe una tabla en el sistema que tiene una entrada por cada *frame* físico, indicando si está libre o no, y si está asignado, a que página de que proceso.

Otra característica para destacar de la paginación es la posibilidad de compartir páginas con el fin de obtener un mejor aprovechamiento de la memoria con lo que se puede lograr una sensible baja en el grado de ocupación de la memoria. Esta compartición consiste en almacenar en un solo lugar un código o datos que pueden ser compartidos por varios usuarios a la vez.

- Detalle 1

Teniendo presentes las características de un sistema paginado, cómo se determinaría el tamaño de la dirección lógica de memoria y del registro de la dirección física de memoria si se tiene, por ejemplo, un espacio de direcciones lógicas de 8 páginas de 2048 bytes cada una, que aloja palabras de 16 bits, con un espacio de direcciones físicas de 32 marcos.

Puesto que en cada página se pueden alojar 2048 bytes y como cada palabra es de 2 bytes (16 bits) entonces se tiene que en una página caben 1024 palabras. Luego, como $\log_2 1024 = 10$ se tiene que se necesitan 10 bits para direccionar 1024 palabras. Además, son necesarios 3 bits para direccionar las 8 páginas porque $\log_2 8 = 3$. Por lo tanto, dirección lógica es de 13 bits.

Con respecto al tamaño del registro de la dirección física de memoria como son 32 los marcos de página, y $\log_2 32 = 5$, resulta entonces que se necesitan 5 bits para direccionar los 32 marcos y 10 para el desplazamiento porque en cada página entran 1024 palabras. Por lo tanto, el registro de direcciones físicas necesita tener 15 bits.

- Detalle 2

Como se sabe, la paginación trabaja en multiprogramación. En tal caso, si el sistema de paginación trabaja en una memoria principal con marcos de página que pueden direccionar 1024 palabras de 64 bits y en la memoria principal deben residir al mismo tiempo los procesos A, B, C, D, E, F y G con los requerimientos de memoria que se indican seguidamente, el sistema de gestión de memoria deberá determinar cuántos marcos necesita para cada proceso y así determinar también la fragmentación interna.

Según los datos que se tienen de la memoria entonces resulta que cada marco de página tiene una capacidad de 8 Kbytes al contener 1024 palabras de 8 bytes cada una.

Luego, para calcular la memoria requerida en cantidad de marcos bastará con dividir la memoria que cada proceso necesita por 8 redondeando el resultado obtenido al siguiente entero cuando la parte fraccionaria del resultado no es nula.

Esa parte fraccionaria no nula permitirá calcular el tamaño de la fragmentación interna. Por ejemplo, si al proceso A se le asignan 157 marcos del último de ellos sólo utilizará el 25% por lo que el 75% de ese marco es fragmentación interna. Es decir que, de los 8 Kbytes del último marco, 6 Kbytes son de fragmentación interna:

Proceso	Memoria requerida en Kbytes	Memoria requerida en cantidad de marcos	Tamaño de la fragmentación en Kbytes
A	1250	156,25 $\Rightarrow$ 157	6
B	6241	780,125 $\Rightarrow$ 781	7
C	2603	325,375 $\Rightarrow$ 326	5

D	2080	260	⇒ 260	0
E	5487	685,875	⇒ 686	1
F	3658	457,25	⇒ 458	6
G	8952	1119	⇒ 1119	0

Segmentación

Es un esquema que apoya la visión que tiene el programador de la memoria. A diferencia de la paginación, los segmentos no tienen una capacidad o tamaño fijo como lo tienen las páginas, sino que su longitud depende del segmento de programa que se vaya a almacenar.

Ahora el proceso del usuario especifica una dirección lógica de dos elementos, uno corresponde al nombre, o número, de segmento y el otro al desplazamiento dentro del segmento. Con ellos se accederá a la tabla de segmentos que contiene en cada una de sus líneas la dirección de base del segmento y la longitud límite de este.

Es posible guardar la tabla de segmentos tanto en un banco de registros rápidos como en memoria principal. La elección estará condicionada por el tamaño de la tabla. Si es pequeña bastará con el banco de registros, pero, si es muy grande deberá ser guardada en la memoria y no en registros, debiéndose en tal caso guardar en un registro la dirección de base de la tabla de segmentos (STBR).

Otras de sus características a destacar son:

- Aunque puede tener fragmentación externa, no tendrá fragmentación interna, y se podrá compactar con cierta facilidad.
- No permite implementar eficientemente memoria virtual.
- La protección está asociada al segmento en lugar de a la dirección, y es posible compartir código o datos.

Por ejemplo, en un sistema de administración de memorias por segmentación se tiene la siguiente tabla de segmentos:

Segmento	Dirección de Base	Desplazamiento Límite
0	219	600
1	2300	14
2	90	100
3	1327	580

4	1952	96
---	------	----

Luego, las direcciones físicas correspondientes a cada una de las direcciones lógicas válidas (porque su número de segmento y su desplazamiento lo son) suministradas por el proceso en ejecución serían las siguientes:

Dirección lógica		Dirección física = Dirección de base + desplazamiento
Segmento	Desplazamiento	
0	250	469 = 219 + 250
1	12	2312 = 2300 + 12
2	85	175 = 90 + 85
3	410	1737 = 1327 + 410
4	82	2034 = 1952 + 82

Por su parte la segmentación paginada se caracteriza por:

- Permite combinar los esquemas de paginación y de segmentación.
- Ser utilizado para no desperdiciar memoria debido a la fragmentación externa que produce la segmentación y además por disminuir el tiempo de búsqueda para asignar un segmento.
- Porque al paginar los segmentos se elimina la fragmentación externa.
- Cada fila en la tabla de segmentos contiene la dirección base de una tabla de páginas para ese segmento.
- El direccionamiento en este caso requerirá de un número o nombre del segmento, un número de página y un desplazamiento dentro de esa página.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.